COURSE ADVANCED DATA SCIENCE LAB | CODE MCA33PE21 | ASSIGNMENT 06

Advanced Data Science Lab [MCA33PE21]

Practical Assignment - 06

K-Means Clustering, K-Nearest Neighbors, Naive Bayes and GridSearchCV

Field	Details
Academic Year	**2026-2027**
Class / Division	**SYMCA (Semester-I)**
PRN	**125M1H064**
Student Name	**Manthan Vijay Sankpal**
Submission Date	**7th of October 2026**

Contents

Task	Algorithm	Dataset
1(a)	K-Means Clustering – customer segmentation	Mall Customer Segmentation
1(b)	K-Nearest Neighbors – iris species classification	Iris
1(c)	Naive Bayes – SMS spam detection (imbalanced data)	SMS Spam Collection
2(a)	GridSearchCV – hyper-parameter tuning of the KNN model	Iris

All random components use `random_state=42`, so every number below is reproducible.

In [1]:
```
# 0. Import all libraries used in this assignment ------------------------------------------------
import warnings
warnings.filterwarnings("ignore")                    # keep the output readable

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay, silhouette_score, average_precision_score, PrecisionRecallDisplay,
)
from sklearn.model_selection import (
    train_test_split, StratifiedKFold, GridSearchCV, cross_val_score, cross_val_predict,
)
from sklearn.naive_bayes import MultinomialNB, ComplementNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

Task 1(a) – K-Means Clustering: Customer Segmentation

Goal: group mall customers by **Annual Income** and **Spending Score** so the mall can target each segment differently. K-Means is *unsupervised*, so there is no train/test split.

1. Load the Dataset and Inspect It

In [2]:
```
mall = pd.read_csv("Mall_Customers.csv")
print("Shape:", mall.shape)
display(mall.head())
```

```
Shape: (200, 5)
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

In [3]:
```
mall.info()
display(mall.describe().round(2))

print("\nMissing values per column:")
print(mall.isnull().sum())
print("Duplicate rows:", mall.duplicated().sum())
print("\nGender split:", mall["Genre"].value_counts().to_dict())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype 
---  ------                  --------------  ----- 
 0   CustomerID              200 non-null    int64 
 1   Genre                   200 non-null    object
 2   Age                     200 non-null    int64 
 3   Annual Income (k$)      200 non-null    int64 
 4   Spending Score (1-100)  200 non-null    int64 
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.00	200.00	200.00	200.00
mean	100.50	38.85	60.56	50.20
std	57.88	13.97	26.26	25.82
min	1.00	18.00	15.00	1.00
25%	50.75	28.75	41.50	34.75
50%	100.50	36.00	61.50	50.00
75%	150.25	49.00	78.00	73.00
max	200.00	70.00	137.00	99.00

```
Missing values per column:
CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
Duplicate rows: 0

Gender split: {'Female': 112, 'Male': 88}
```

2. Data Preprocessing

- **Missing values:** none were found, so nothing needs to be imputed.
- **Feature selection:** as required, only **Annual Income (k$)** and **Spending Score (1-100)** are used.
- **Scaling:** K-Means is distance-based, so features on larger scales dominate the distance. `StandardScaler` (mean 0, std 1) puts both features on an equal footing.

In [4]:
```
features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = mall[features]

# Quick look at the two selected features
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(X[features[0]], kde=True, ax=axes[0], color="#1d7a8c")
sns.histplot(X[features[1]], kde=True, ax=axes[1], color="#d1495b")
axes[2].scatter(X[features[0]], X[features[1]], alpha=0.7, color="gray")
axes[0].set_title("Annual income distribution"); axes[1].set_title("Spending score distribution")
axes[2].set(title="Raw data (no labels yet)", xlabel=features[0], ylabel=features[1])
plt.tight_layout(); plt.show()

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Scaled means:", X_scaled.mean(axis=0).round(3), "| scaled std:", X_scaled.std(axis=0).round(3))
```




```
Scaled means: [-0. -0.] | scaled std: [1. 1.]
```

3. Elbow Method – Choosing the Optimal k

The **inertia** (within-cluster sum of squared distances) always falls as k grows. The *elbow* is the k after which adding clusters gives only small gains. To back up the visual choice, two extra checks are shown: the **silhouette score** (higher = better separated clusters) and the point that lies **farthest from the straight line** joining the first and last inertia values (a simple automatic elbow detector).

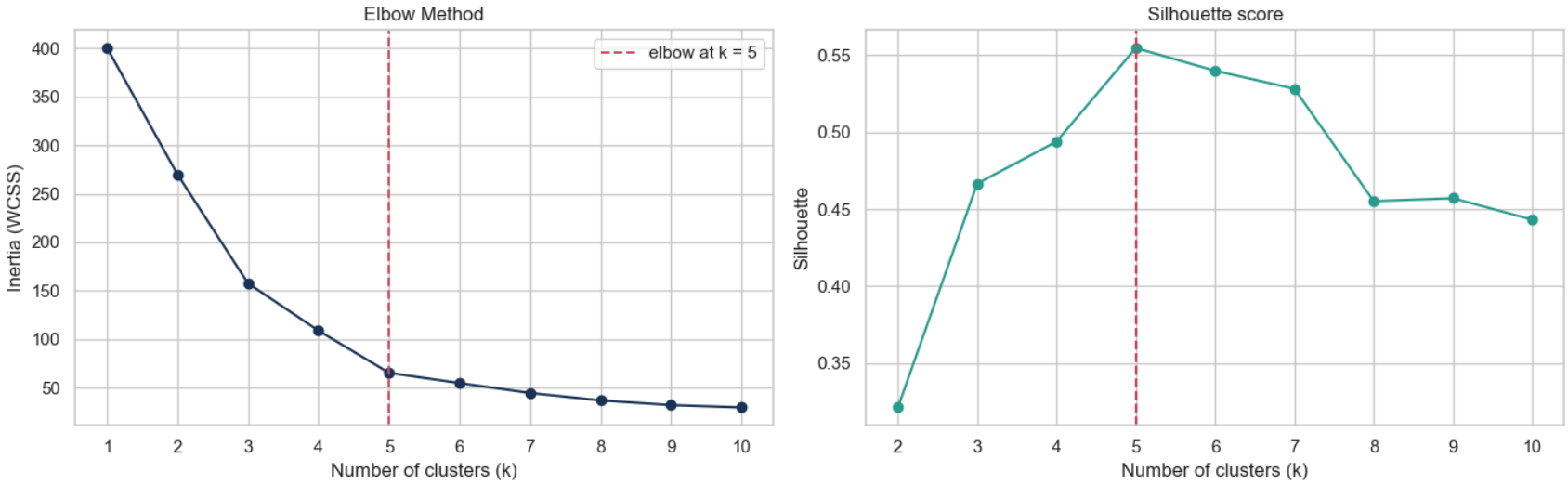
In [5]:
```
k_range = range(1, 11)
inertia, silhouette = [], {}
for k in k_range:
    km = KMeans(n_clusters=k, n_init=10, random_state=RANDOM_STATE).fit(X_scaled)
    inertia.append(km.inertia_)
    if k > 1:
```

```
        silhouette[k] = silhouette_score(X_scaled, km.labels_)

# Elbow detector: distance of each (k, inertia) point from the chord joining the first and last point
ks = np.array(list(k_range))
(x1, y1), (x2, y2) = (ks[0], inertia[0]), (ks[-1], inertia[-1])
dist = [abs((x2 - x1) * (y1 - y) - (x1 - x) * (y2 - y1)) / np.hypot(x2 - x1, y2 - y1) for x, y in zip(ks, inertia)]
optimal_k = int(ks[int(np.argmax(dist))])

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
axes[0].plot(ks, inertia, marker="o", color="#1d3557")
axes[0].axvline(optimal_k, color="#d1495b", ls="--", label=f"elbow at k = {optimal_k}")
axes[0].set(title="Elbow Method", xlabel="Number of clusters (k)", ylabel="Inertia (WCSS)", xticks=ks)
axes[0].legend()
axes[1].plot(list(silhouette), list(silhouette.values()), marker="o", color="#2a9d8f")
axes[1].axvline(optimal_k, color="#d1495b", ls="--")
axes[1].set(title="Silhouette score", xlabel="Number of clusters (k)", ylabel="Silhouette", xticks=ks[1:])
plt.tight_layout(); plt.show()

print("Inertia by k:", {int(k): round(v, 1) for k, v in zip(ks, inertia)})
print("Silhouette by k:", {k: round(v, 3) for k, v in silhouette.items()})
print(f"\nOptimal number of clusters chosen: k = {optimal_k}")
```




```
Inertia by k: {1: 400.0, 2: 269.7, 3: 157.7, 4: 108.9, 5: 65.6, 6: 55.1, 7: 44.9, 8: 37.2, 9: 32.4, 10: 30.0}
Silhouette by k: {2: 0.321, 3: 0.467, 4: 0.494, 5: 0.555, 6: 0.54, 7: 0.528, 8: 0.455, 9: 0.457, 10: 0.443}

Optimal number of clusters chosen: k = 5
```

4. Train the Final K-Means Model and Report the Inertia

In [6]:
```
kmeans = KMeans(n_clusters=optimal_k, n_init=10, random_state=RANDOM_STATE)
mall["Cluster"] = kmeans.fit_predict(X_scaled)

# Centroids back in the original units (k$ and score) so they can be read directly
centroids = scaler.inverse_transform(kmeans.cluster_centers_)

# Inertia in original units, for interpretation (sum of squared distances to the own centroid)
inertia_original = ((X.values - centroids[mall["Cluster"]]) ** 2).sum()

print(f"Final model inertia (scaled features):   {kmeans.inertia_:.3f}")
print(f"Same inertia in original units:          {inertia_original:,.1f}")
print(f"Silhouette score of the final clustering: {silhouette_score(X_scaled, kmeans.labels_):.3f}")
```

```
Final model inertia (scaled features):   65.568
Same inertia in original units:          44,448.5
Silhouette score of the final clustering: 0.555
```

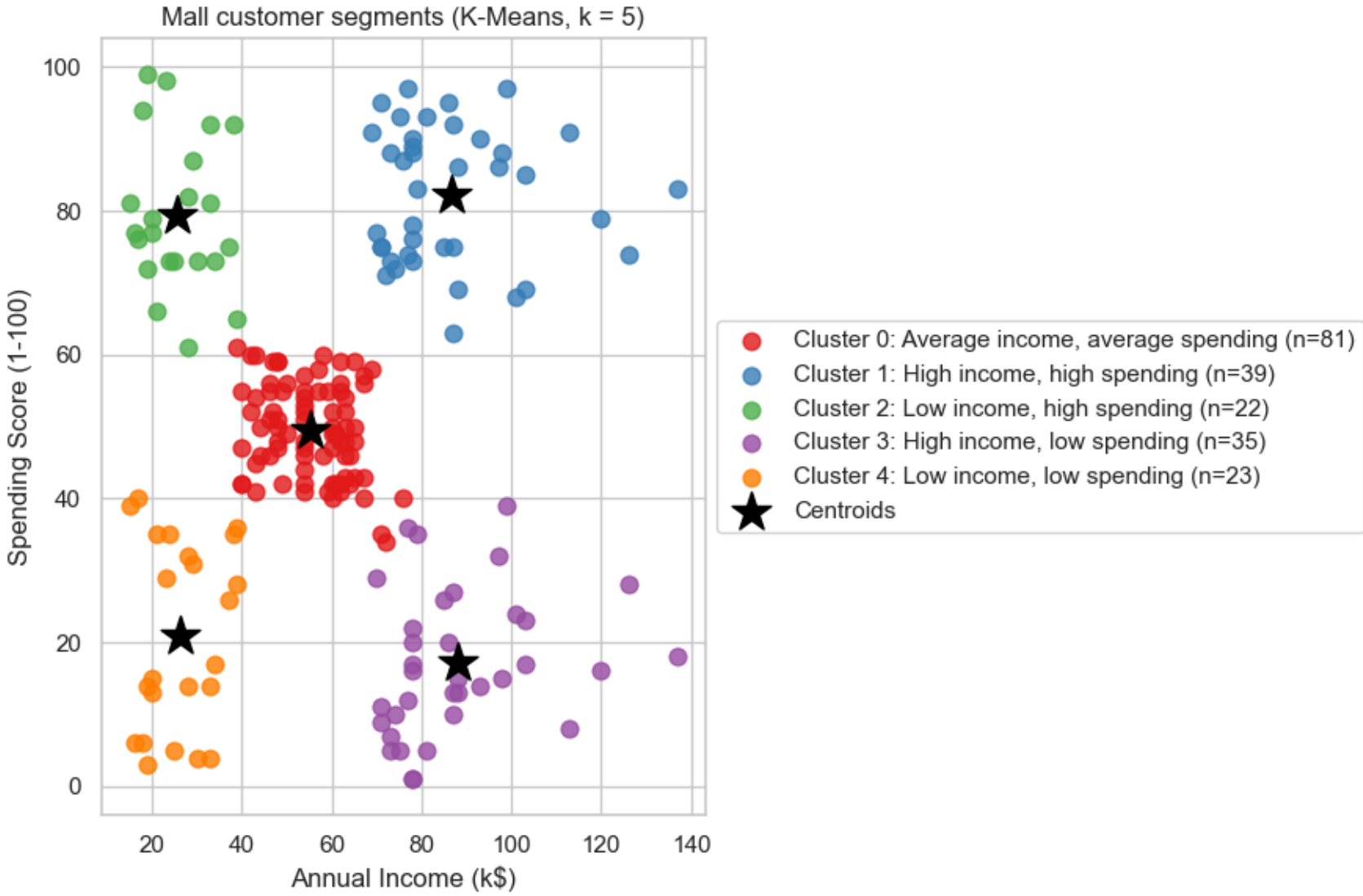
5. Visualise the Clusters

In [7]:
```
# Name each cluster from where its centroid lies (z-score > 0 = above average)
def persona(z_income, z_spend):
    if abs(z_income) < 0.5 and abs(z_spend) < 0.5:
        return "Average income, average spending"
    return {(True, True): "High income, high spending",
            (True, False): "High income, low spending",
            (False, True): "Low income, high spending",
            (False, False): "Low income, low spending"}[(z_income > 0, z_spend > 0)]

names = {c: persona(*kmeans.cluster_centers_[c]) for c in range(optimal_k)}
palette = sns.color_palette("Set1", optimal_k)

plt.figure(figsize=(9, 6))
for c in range(optimal_k):
    pts = mall[mall["Cluster"] == c]
    plt.scatter(pts[features[0]], pts[features[1]], s=60, color=palette[c], alpha=0.8,
                label=f"Cluster {c}: {names[c]} (n={len(pts)})")
plt.scatter(centroids[:, 0], centroids[:, 1], s=350, c="black", marker="*", label="Centroids")
plt.title(f"Mall customer segments (K-Means, k = {optimal_k})")
```

```
plt.xlabel("Annual Income (k$)"); plt.ylabel("Spending Score (1-100)")
plt.legend(loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout(); plt.show()
```



6. Cluster Profiles

In [8]:
```
profile = (mall.groupby("Cluster")
           .agg(Customers=("Cluster", "size"),
                Avg_Income=(features[0], "mean"),
                Avg_Spending=(features[1], "mean"),
                Avg_Age=("Age", "mean"),
                Female_pct=("Genre", lambda s: (s == "Female").mean() * 100))
           .round(1))
profile["Segment"] = pd.Series(names)
profile.sort_values("Avg_Spending", ascending=False)
```

Out[8]:

Cluster	Customers	Avg_Income	Avg_Spending	Avg_Age	Female_pct	Segment
1	39	86.5	82.1	32.7	53.8	High income, high spending
2	22	25.7	79.4	25.3	59.1	Low income, high spending
0	81	55.3	49.5	42.7	59.3	Average income, average spending
4	23	26.3	20.9	45.2	60.9	Low income, low spending
3	35	88.2	17.1	41.1	45.7	High income, low spending

7. Findings and Insights

- **k = 5 is the right number of clusters.** The inertia curve bends sharply at k = 5 (inertia 65.6) and improves only slowly afterwards; the automatic elbow detector also picks 5, and the **silhouette score peaks at k = 5 (0.555)**.
- **Final model inertia = 65.57** on the scaled features (≈ 44,448 in original k$/score units); the silhouette of 0.555 indicates well-separated, compact clusters.
- **Five meaningful customer segments:**

Segment	Customers	Avg. income (k$)	Avg. spending score	Avg. age	Suggested action
High income, high spending	39	86.5	82.1	32.7	**Primary target** – premium offers, loyalty programmes
Low income, high spending	22	25.7	79.4	25.3	Young, enthusiastic – trendy / discount offers

Segment	Customers	Avg. income (k$)	Avg. spending score	Avg. age	Suggested action
Average income, average spending	81	55.3	49.5	42.7	Largest group – steady, mainstream promotions
High income, low spending	35	88.2	17.1	41.1	Under-served – personalised campaigns to unlock spending
Low income, low spending	23	26.3	20.9	45.2	Price-sensitive – low marketing priority

- **Income alone does not predict spending**: customers with similar high incomes split into big spenders (86.5 k $, score 82) and very frugal ones ($88.2k$, score 17) – exactly why a two-feature clustering is more useful than income bands.
- The two high-spending groups are the **youngest** (average age 25 and 33), the low-spending groups the oldest (41–45).
- *Limitation:* K-Means assumes roughly round clusters and needs k in advance; here the data are visibly compact blobs, so it works very well.

Task 1(b) – K-Nearest Neighbors: Iris Classification

Goal: predict the species (*setosa, versicolor, virginica*) from sepal and petal measurements.

1. Load the Dataset and Inspect It

In [9]:
```
iris = pd.read_csv("Iris.csv").drop(columns="Id")       # 'Id' is only a row number, not a feature
print("Shape:", iris.shape)
display(iris.head())
display(iris.describe().round(2))
print("Missing values:", int(iris.isnull().sum().sum()), "| Duplicate rows:", int(iris.duplicated().sum()))
print("\nClass balance:", iris["Species"].value_counts().to_dict())
```

```
Shape: (150, 5)
```

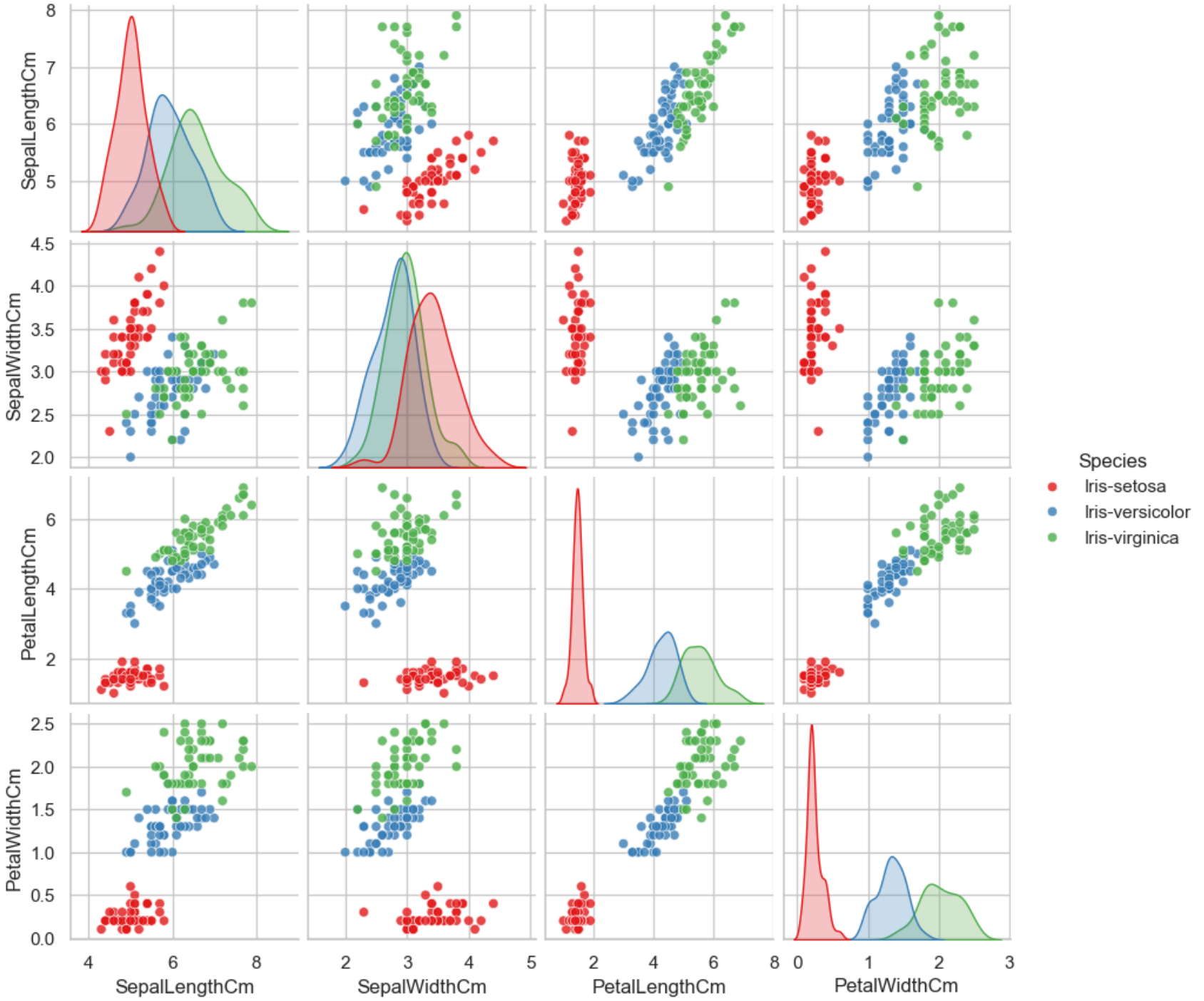
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.00	150.00	150.00	150.00
mean	5.84	3.05	3.76	1.20
std	0.83	0.43	1.76	0.76
min	4.30	2.00	1.00	0.10
25%	5.10	2.80	1.60	0.30
50%	5.80	3.00	4.35	1.30
75%	6.40	3.30	5.10	1.80
max	7.90	4.40	6.90	2.50

```
Missing values: 0 | Duplicate rows: 3

Class balance: {'Iris-setosa': 50, 'Iris-versicolor': 50, 'Iris-virginica': 50}
```

In [10]:
```
# Exploratory plot: which measurements separate the species?
sns.pairplot(iris, hue="Species", palette="Set1", height=2.2, plot_kws={"alpha": 0.8})
plt.show()
```


SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica

The pair plot shows **setosa** is completely separated from the other two species (especially by petal length / width), while **versicolor** and **virginica** overlap slightly – that overlap is where any classification errors will come from.

2. Data Preprocessing and Train/Test Split

- **Missing values:** none.
- **Split:** 80 % training / 20 % testing, **stratified** so each species keeps the same share in both sets.
- **Scaling:** KNN is distance-based, so features are standardised. The scaler is **fitted on the training data only** and then applied to the test data, so no test information leaks into training.

In [11]:
```
X_iris = iris.drop(columns="Species")
y_iris = iris["Species"]

X_train, X_test, y_train, y_test = train_test_split(
    X_iris, y_iris, test_size=0.20, random_state=RANDOM_STATE, stratify=y_iris
)
print("Training samples:", len(X_train), "| Testing samples:", len(X_test))
print("Test class counts:", y_test.value_counts().to_dict())

iris_scaler = StandardScaler().fit(X_train)             # learn mean / std from training data only
X_train_s = iris_scaler.transform(X_train)
X_test_s = iris_scaler.transform(X_test)
```

```
Training samples: 120 | Testing samples: 30
Test class counts: {'Iris-setosa': 10, 'Iris-virginica': 10, 'Iris-versicolor': 10}
```

3. Implement KNN, Train and Predict (default model, k = 5)

In [12]:
```
knn = KNeighborsClassifier()                            # defaults: n_neighbors=5, weights='uniform', Euclidean dista
knn.fit(X_train_s, y_train)
y_pred = knn.predict(X_test_s)

comparison = pd.DataFrame({"Actual": y_test.values, "Predicted": y_pred}, index=y_test.index)
display(comparison.head(10))
```

	Actual	Predicted
38	Iris-setosa	Iris-setosa
127	Iris-virginica	Iris-virginica
57	Iris-versicolor	Iris-versicolor
93	Iris-versicolor	Iris-versicolor
42	Iris-setosa	Iris-setosa
56	Iris-versicolor	Iris-versicolor
22	Iris-setosa	Iris-setosa
20	Iris-setosa	Iris-setosa
147	Iris-virginica	Iris-virginica
84	Iris-versicolor	Iris-versicolor

4. Model Evaluation

In [13]:
```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="macro")
rec = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")
default_metrics = {"Accuracy": acc, "Precision": prec, "Recall": rec, "F1-score": f1}   # kept for the comparison in

print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}  (macro average over the 3 classes)")
print(f"Recall   : {rec:.4f}  (macro average)")
print(f"F1-score : {f1:.4f}  (macro average)")

print("\nClassification report:\n")
print(classification_report(y_test, y_pred))

print("Confusion matrix (rows = actual, columns = predicted):")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy : 0.9333
Precision: 0.9444  (macro average over the 3 classes)
Recall   : 0.9333  (macro average)
F1-score : 0.9327  (macro average)

Classification report:

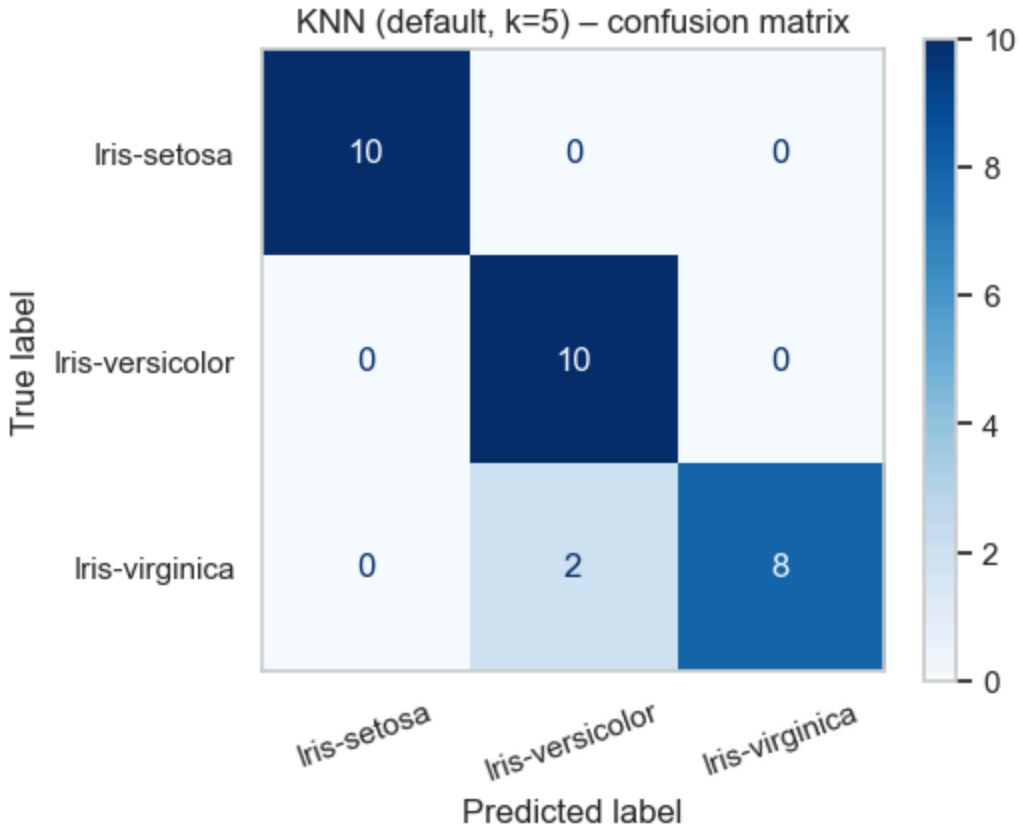
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       0.83      1.00      0.91        10
 Iris-virginica       1.00      0.80      0.89        10

       accuracy                           0.93        30
      macro avg       0.94      0.93      0.93        30
   weighted avg       0.94      0.93      0.93        30

Confusion matrix (rows = actual, columns = predicted):
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
```

In [14]:
```
fig, ax = plt.subplots(figsize=(5.5, 4.5))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap="Blues", ax=ax, xticks_rotation=20)
ax.set_title("KNN (default, k=5) – confusion matrix")
ax.grid(False)
plt.tight_layout(); plt.show()
```



5. Findings and Insights

- The default KNN reaches **93.3 % accuracy** (28 of 30 test flowers) with macro precision 0.944, recall 0.933 and F1 0.933.
- **Setosa is classified perfectly** (precision and recall 1.00) and **every versicolor is found** (recall 1.00). The only errors are **2 virginica flowers predicted as versicolor** (virginica recall 0.80, versicolor precision 0.83) – the two species whose measurements overlap in the pair plot.
- With only 30 test flowers, **one mistake = 3.3 percentage points of accuracy**, so small differences between models on this test set should be interpreted cautiously (cross-validation in Task 2(a) gives a steadier estimate).
- KNN needs **scaling** (done) and has no training phase – it stores the data and votes among the k nearest neighbours, so the choice of k and the weighting matter; these are tuned next.

Task 1(c) – Naive Bayes Classification: SMS Spam Detection

Goal: classify an SMS as **ham** (legitimate) or **spam**. The classes are imbalanced, so *accuracy alone is misleading* – the key metrics are the **precision and recall of the spam class**:

- **Precision (spam)** – of the messages flagged as spam, how many really are (low precision = legitimate messages get blocked).
- **Recall (spam)** – of all real spam, how much is caught (low recall = spam reaches the inbox).

1. Load the Dataset and Inspect It

In [15]:
```
sms = pd.read_csv("SMSSpamCollection", sep="\t", header=None, names=["label", "text"], encoding="latin-1")
print("Shape:", sms.shape)
display(sms.head())

print("Missing values:", int(sms.isnull().sum().sum()))
print("Duplicate messages:", int(sms.duplicated().sum()))
print("Class balance:", sms["label"].value_counts().to_dict())
```

```
Shape: (5572, 2)
```

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
Missing values: 0
Duplicate messages: 403
Class balance: {'ham': 4825, 'spam': 747}
```

Handling duplicates. 403 messages appear more than once. If the same text lands in both the training and the test set, the test score is inflated (the model has "seen the answer"), so exact duplicates are removed before splitting.

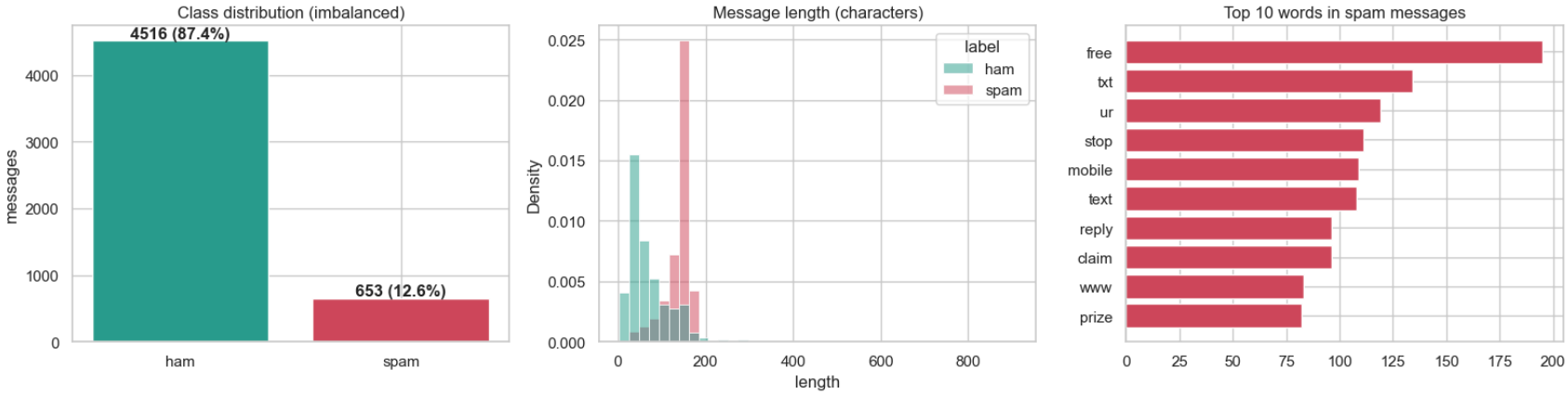
In [16]:
```
sms = sms.drop_duplicates().reset_index(drop=True)
counts = sms["label"].value_counts()
print("After removing duplicates:", sms.shape[0], "messages")
print(f"ham: {counts['ham']} ({counts['ham'] / len(sms):.1%})   spam: {counts['spam']} ({counts['spam'] / len(sms):.1

fig, axes = plt.subplots(1, 3, figsize=(16, 4.2))
axes[0].bar(counts.index, counts.values, color=["#2a9d8f", "#d1495b"])
for i, n in enumerate(counts.values):
    axes[0].text(i, n + 40, f"{n} ({n / len(sms):.1%})", ha="center", fontweight="bold")
axes[0].set(title="Class distribution (imbalanced)", ylabel="messages")

sms["length"] = sms["text"].str.len()
sns.histplot(data=sms, x="length", hue="label", palette={"ham": "#2a9d8f", "spam": "#d1495b"},
             bins=40, stat="density", common_norm=False, alpha=0.5, ax=axes[1])
axes[1].set(title="Message length (characters)")

def top_words(label, n=10):
    vec = CountVectorizer(stop_words="english")
    counts_ = vec.fit_transform(sms.loc[sms["label"] == label, "text"]).sum(axis=0).A1
    return pd.Series(counts_, index=vec.get_feature_names_out()).nlargest(n).sort_values()
spam_words = top_words("spam")
axes[2].barh(spam_words.index, spam_words.values, color="#d1495b")
axes[2].set(title="Top 10 words in spam messages")
plt.tight_layout(); plt.show()
print("Average length:", sms.groupby("label")["length"].mean().round(0).to_dict())
print("Top ham words :", list(top_words("ham").index[::-1]))
```

```
After removing duplicates: 5169 messages
ham: 4516 (87.4%)   spam: 653 (12.6%)
```



```
Average length: {'ham': 71.0, 'spam': 138.0}
Top ham words : ['gt', 'lt', 'just', 'ok', 'll', 'got', 'know', 'like', 'good', 'come']
```

EDA insight: spam messages are about **twice as long** (138 vs. 71 characters on average) and are full of call-to-action words – *free, txt, stop, mobile, claim, reply, prize*. Ham messages use everyday conversational words (*ok, just, got, know, good, come*). This vocabulary difference is exactly what Naive Bayes exploits.

2. Data Preprocessing and Train/Test Split

- **Label encoding:** ham → 0, spam → 1.
- **Text → numbers:** `CountVectorizer` turns each message into word counts (a bag of words). The vectoriser is part of a `Pipeline`, so it is **fitted on training messages only**.
- **Split:** 80 / 20, **stratified** so both sets keep ≈ 12.6 % spam.
- **Model choice:** `MultinomialNB` is the natural Naive Bayes for word counts. `GaussianNB` assumes continuous, bell-shaped features and is not suited to sparse count data.

In [17]:
```
y_sms = (sms["label"] == "spam").astype(int)            # ham = 0, spam = 1
X_tr, X_te, y_tr, y_te = train_test_split(
    sms["text"], y_sms, test_size=0.20, random_state=RANDOM_STATE, stratify=y_sms
)
print(f"Training messages: {len(X_tr)} (spam {y_tr.mean():.1%})  |  Testing messages: {len(X_te)} (spam {y_te.mean()
```

```
Training messages: 4135 (spam 12.6%)  |  Testing messages: 1034 (spam 12.7%, 131 spam)
```

3. Baseline Model: CountVectorizer + MultinomialNB

In [18]:
```
def spam_scores(y_true, y_pred):
    """Metrics that matter for an imbalanced problem (spam = positive class)."""
    cm = confusion_matrix(y_true, y_pred)
    return {"Accuracy": accuracy_score(y_true, y_pred),
            "Spam precision": precision_score(y_true, y_pred),
            "Spam recall": recall_score(y_true, y_pred),
            "Spam F1": f1_score(y_true, y_pred),
            "Missed spam (FN)": int(cm[1, 0]),
            "Ham blocked (FP)": int(cm[0, 1])}
```

```
baseline = Pipeline([("vec", CountVectorizer()), ("nb", MultinomialNB())])
baseline.fit(X_tr, y_tr)
y_pred_base = baseline.predict(X_te)

print("Classification report – baseline:\n")
print(classification_report(y_te, y_pred_base, target_names=["ham", "spam"], digits=3))
print("Confusion matrix (rows = actual, columns = predicted):")
print(confusion_matrix(y_te, y_pred_base))
```

```
Classification report – baseline:

              precision    recall  f1-score   support

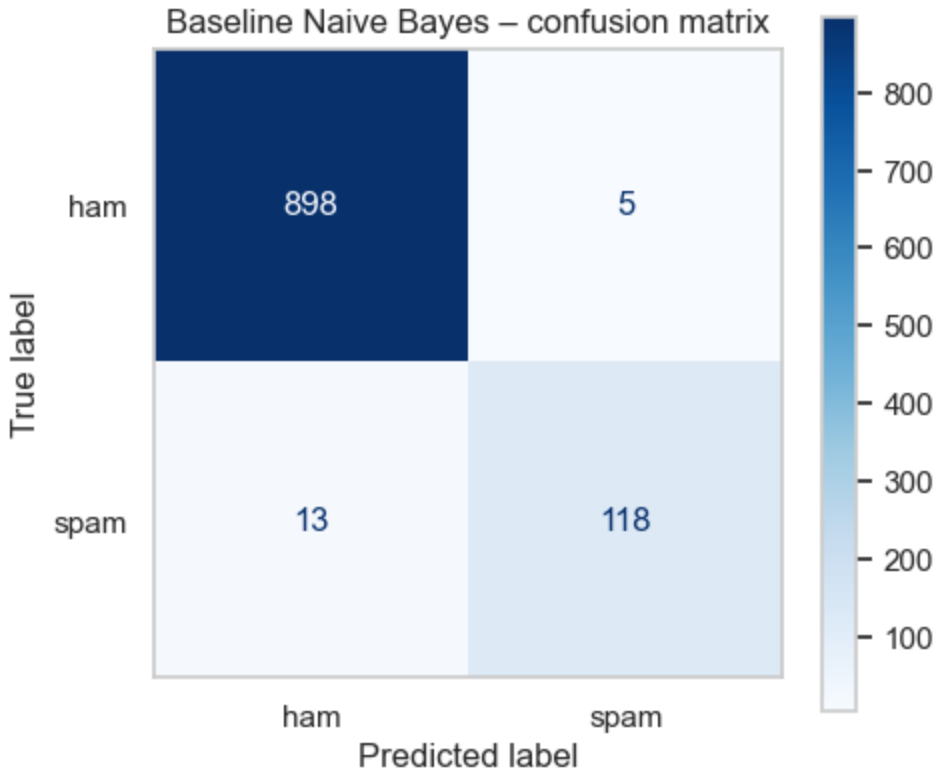
         ham      0.986     0.994     0.990       903
        spam      0.959     0.901     0.929       131

    accuracy                          0.983      1034
   macro avg      0.973     0.948     0.960      1034
weighted avg      0.982     0.983     0.982      1034

Confusion matrix (rows = actual, columns = predicted):
[[898   5]
 [ 13 118]]
```

In [19]:

```
fig, ax = plt.subplots(figsize=(5, 4.3))
ConfusionMatrixDisplay.from_predictions(y_te, y_pred_base, display_labels=["ham", "spam"], cmap="Blues", ax=ax)
ax.set_title("Baseline Naive Bayes – confusion matrix"); ax.grid(False)
plt.tight_layout(); plt.show()
```



Baseline reading: accuracy looks excellent (98.3 %), but this is partly because 87 % of messages are ham. The spam-specific scores tell the real story: precision 0.959 but **recall only 0.901 – 13 of the 131 spam messages reach the inbox** – and 5 legitimate messages are wrongly blocked.

4. Improving the Scores for the Imbalanced Dataset

Several standard techniques are compared. Each is trained on the training set only and evaluated on the same untouched test set:

Code	Technique	Idea
A	Baseline	CountVectorizer + MultinomialNB (α = 1, class prior learned from the data)
B	TF-IDF features	down-weights very common words
C	Uniform class prior (`fit_prior=False`)	stops the model from favouring the majority class 'ham' by default

Code	Technique	Idea
D	Random over-sampling of spam	duplicate spam messages in the **training set only** until both classes are equal
E	`ComplementNB`	Naive Bayes variant designed for imbalanced text
F	**GridSearchCV** over vectoriser, n-grams, stop-words, α and prior	tuned for the F1-score of the spam class (5-fold CV on training data)
G	F + **decision-threshold tuning**	threshold chosen from cross-validated (out-of-fold) training predictions to maximise spam F1

In [20]:
```
results = {}
results["A  Baseline (Count + MNB)"] = spam_scores(y_te, y_pred_base)

# B - TF-IDF features
tfidf = Pipeline([("vec", TfidfVectorizer()), ("nb", MultinomialNB())]).fit(X_tr, y_tr)
results["B  TF-IDF + MNB"] = spam_scores(y_te, tfidf.predict(X_te))

# C - uniform class prior
uniform = Pipeline([("vec", CountVectorizer()), ("nb", MultinomialNB(fit_prior=False))]).fit(X_tr, y_tr)
results["C  Uniform class prior"] = spam_scores(y_te, uniform.predict(X_te))

# D - random over-sampling of the minority class (training data only!)
train_df = pd.DataFrame({"text": X_tr, "y": y_tr})
ham_train, spam_train = train_df[train_df["y"] == 0], train_df[train_df["y"] == 1]
oversampled = pd.concat([ham_train, spam_train.sample(len(ham_train), replace=True, random_state=RANDOM_STATE)])
print(f"Training set after over-sampling: ham = {(oversampled['y'] == 0).sum()}, spam = {(oversampled['y'] == 1).sum(
over = Pipeline([("vec", CountVectorizer()), ("nb", MultinomialNB())]).fit(oversampled["text"], oversampled["y"])
results["D  Over-sampled spam"] = spam_scores(y_te, over.predict(X_te))

# E - Complement Naive Bayes
comp = Pipeline([("vec", CountVectorizer()), ("nb", ComplementNB())]).fit(X_tr, y_tr)
results["E  ComplementNB"] = spam_scores(y_te, comp.predict(X_te))
```

```
Training set after over-sampling: ham = 3613, spam = 3613
```

In [21]:
```
# F - GridSearchCV: vectoriser type, n-grams, min_df, stop-words, smoothing alpha and class prior
cv5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
param_grid = {
    "vec": [CountVectorizer(), TfidfVectorizer()],
```

```
    "vec__ngram_range": [(1, 1), (1, 2)],
    "vec__min_df": [1, 2],
    "vec__stop_words": [None, "english"],
    "nb__alpha": [0.01, 0.1, 0.5, 1.0],
    "nb__fit_prior": [True, False],
}
search = GridSearchCV(Pipeline([("vec", CountVectorizer()), ("nb", MultinomialNB())]),
                      param_grid, cv=cv5, scoring="f1", n_jobs=1)
search.fit(X_tr, y_tr)

print("Combinations searched:", len(search.cv_results_["params"]))
print("Best parameters:", {k: (type(v).__name__ if k == "vec" else v) for k, v in search.best_params_.items()})
print(f"Best cross-validated spam F1: {search.best_score_:.4f}")
results["F  Tuned (GridSearchCV)"] = spam_scores(y_te, search.predict(X_te))
```

```
Combinations searched: 128
Best parameters: {'nb__alpha': 0.1, 'nb__fit_prior': True, 'vec': 'CountVectorizer', 'vec__min_df': 1, 'vec__ngram_ra
nge': (1, 1), 'vec__stop_words': None}
Best cross-validated spam F1: 0.9436
```

In [22]:
```
# G - decision-threshold tuning, chosen on OUT-OF-FOLD training probabilities (the test set is not touched)
best_model = search.best_estimator_
oof_proba = cross_val_predict(best_model, X_tr, y_tr, cv=cv5, method="predict_proba")[:, 1]
thresholds = np.linspace(0.05, 0.95, 91)
oof_f1 = [f1_score(y_tr, oof_proba >= t) for t in thresholds]
best_t = float(thresholds[int(np.argmax(oof_f1))])
print(f"Best threshold on out-of-fold predictions: {best_t:.2f}  (spam F1 = {max(oof_f1):.4f}; default threshold is 0

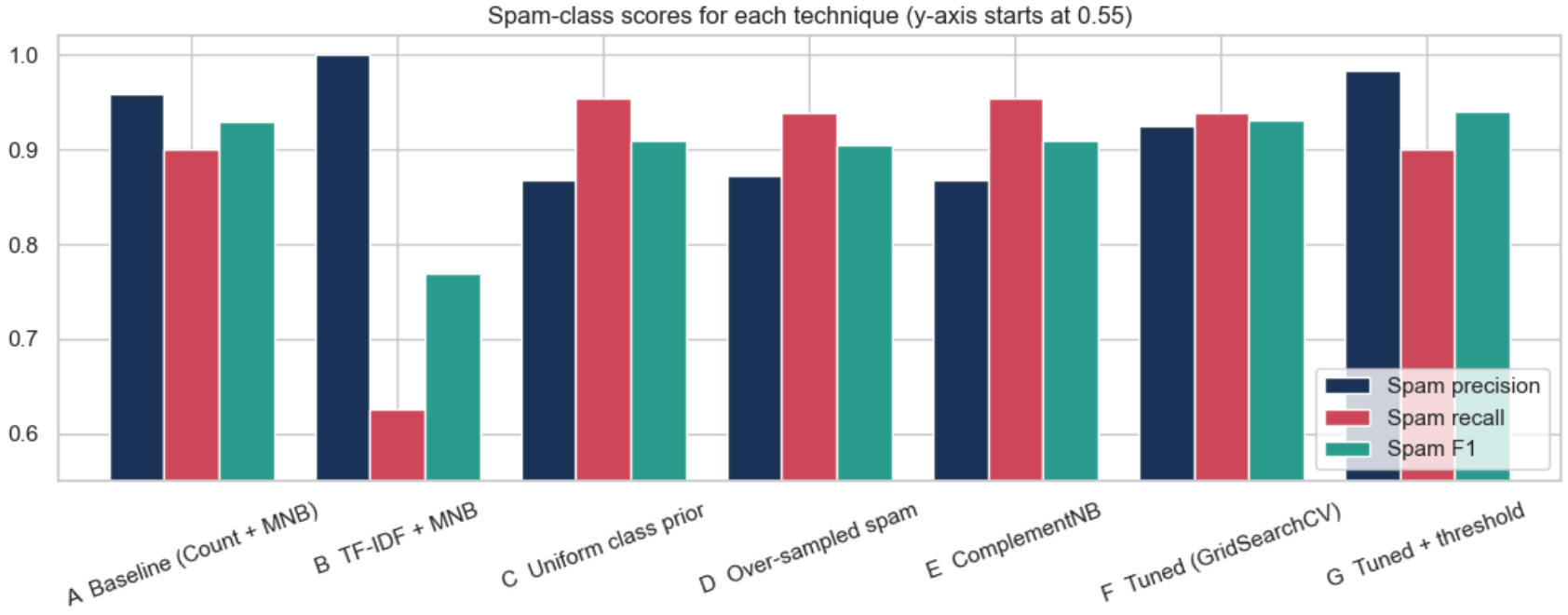
proba_te = best_model.predict_proba(X_te)[:, 1]
y_pred_final = (proba_te >= best_t).astype(int)
results["G  Tuned + threshold"] = spam_scores(y_te, y_pred_final)

summary = pd.DataFrame(results).T.astype({"Missed spam (FN)": int, "Ham blocked (FP)": int})
display(summary.round(4))
```

```
Best threshold on out-of-fold predictions: 0.85  (spam F1 = 0.9496; default threshold is 0.50)
```

	Accuracy	Spam precision	Spam recall	Spam F1	Missed spam (FN)	Ham blocked (FP)
A Baseline (Count + MNB)	0.9826	0.9593	0.9008	0.9291	13	5
B TF-IDF + MNB	0.9526	1.0000	0.6260	0.7700	49	0
C Uniform class prior	0.9758	0.8681	0.9542	0.9091	6	19
D Over-sampled spam	0.9749	0.8723	0.9389	0.9044	8	18
E ComplementNB	0.9758	0.8681	0.9542	0.9091	6	19
F Tuned (GridSearchCV)	0.9826	0.9248	0.9389	0.9318	8	10
G Tuned + threshold	0.9855	0.9833	0.9008	0.9402	13	2

In [23]:
```
# Visual comparison of the spam-class scores
summary[["Spam precision", "Spam recall", "Spam F1"]].plot.bar(figsize=(11, 4.5), rot=20, width=0.8,
                                                               color=["#1d3557", "#d1495b", "#2a9d8f"])
plt.ylim(0.55, 1.02); plt.title("Spam-class scores for each technique (y-axis starts at 0.55)")
plt.legend(loc="lower right"); plt.tight_layout(); plt.show()
```



5. Final Model Evaluation (G: tuned Naive Bayes + tuned threshold)

In [24]:
```
print("Classification report – final model:\n")
print(classification_report(y_te, y_pred_final, target_names=["ham", "spam"], digits=3))
print("Confusion matrix (rows = actual, columns = predicted):")
print(confusion_matrix(y_te, y_pred_final))

fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
ConfusionMatrixDisplay.from_predictions(y_te, y_pred_final, display_labels=["ham", "spam"], cmap="Blues", ax=axes[0])
axes[0].set_title("Final model – confusion matrix"); axes[0].grid(False)
PrecisionRecallDisplay.from_predictions(y_te, baseline.predict_proba(X_te)[:, 1], name="Baseline", ax=axes[1])
PrecisionRecallDisplay.from_predictions(y_te, proba_te, name="Tuned", ax=axes[1])
axes[1].set_title("Precision-recall curve (spam class)")
plt.tight_layout(); plt.show()
```

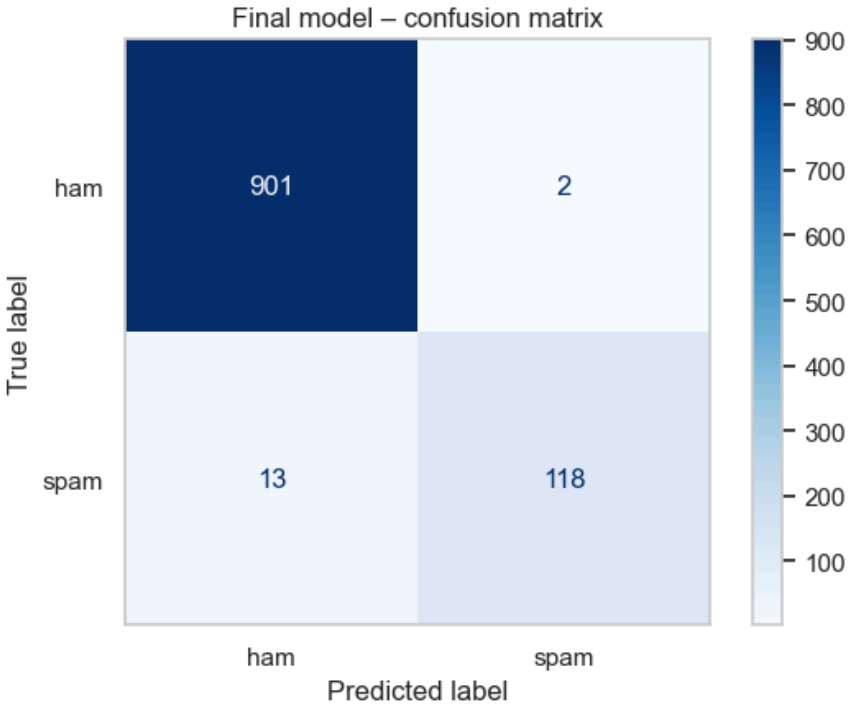
```
Classification report – final model:

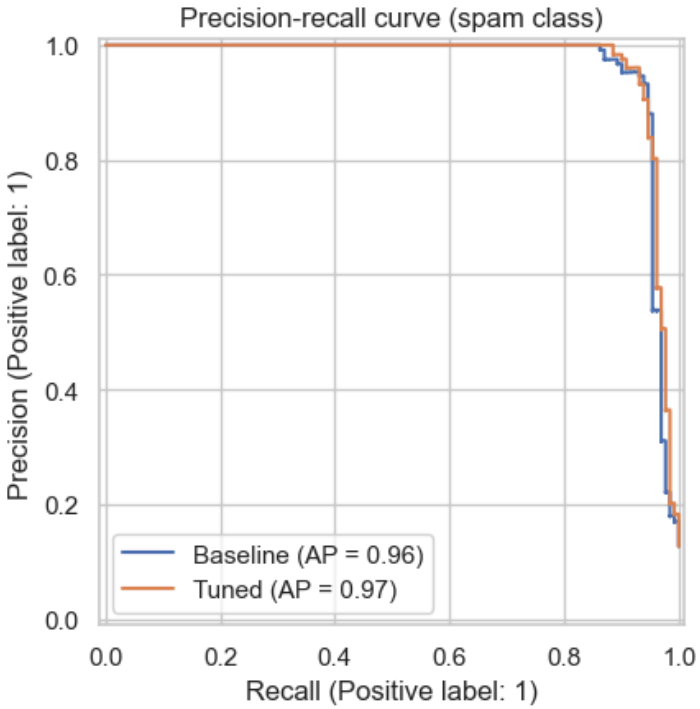
              precision    recall  f1-score   support

         ham      0.986     0.998     0.992       903
        spam      0.983     0.901     0.940       131

    accuracy                          0.985      1034
   macro avg      0.985     0.949     0.966      1034
weighted avg      0.985     0.985     0.985      1034

Confusion matrix (rows = actual, columns = predicted):
[[901   2]
 [ 13 118]]
```





6. Findings and Insights

- **Accuracy is misleading here.** Every technique scores 95–99 % accuracy because 87 % of messages are ham; the precision / recall of the **spam** class separate the models.
- **There is a precision–recall trade-off.** Techniques that tell the model to take spam more seriously – **uniform prior (C)**, **over-sampling (D)** and **ComplementNB (E)** – raise spam recall from 0.901 to 0.94–0.95 (missed spam falls from 13 to 6–8) but precision drops from 0.959 to ≈ 0.87 (**18–19 legitimate messages blocked** instead of 5). Over-sampling and the uniform prior behave almost identically because both remove the model's built-in preference for 'ham'.
- **TF-IDF with default smoothing (B) hurts recall** (0.626, 49 missed spam) – a reminder that Naive Bayes hyper-parameters (especially the smoothing α) must be tuned together with the feature extraction.
- **GridSearchCV (F)** chose plain word counts, unigrams, no stop-word removal, **α = 0.1** and the learned prior; it gives a better-balanced model (precision 0.925, recall 0.939, **F1 0.932** vs. 0.929 baseline).
- **Threshold tuning (G) gives the best overall result:** spam **precision 0.983**, recall 0.901 and **F1 0.940** (the highest), with **only 2 legitimate messages blocked** instead of 5 and the same 13 missed spam as the baseline. The threshold (0.85) was chosen from cross-validated predictions on the training data, so the test score is an honest estimate.

- **Which model to use?** For a real SMS filter, blocking a legitimate message is usually worse than letting some spam through, so **G (precision-focused)** is the best choice; if missing spam is the bigger concern, **C/E** (recall 0.954) is the alternative. The trade-off is a business decision, not a modelling one.
- *Limitation:* the test set has only 131 spam messages, so one message changes recall by 0.8 points – differences of 1–2 messages between the tuned variants are within noise.

Task 2(a) – Hyper-parameter Tuning of KNN with GridSearchCV (Iris)

Goal: improve the default KNN model of Task 1(b) by searching for better hyper-parameters.

1. Parameter Grid, Cross-Validation Strategy and Search

- **Hyper-parameters tuned (three):** `n_neighbors` (1–25), `weights` (`uniform` vs. `distance`) and the distance `metric` (`euclidean` vs. `manhattan`) → 25 × 2 × 2 = **100 combinations**.
- **Cross-validation:** stratified **5-fold** on the training data only (the test set is never used for tuning).
- **Pipeline:** the scaler is inside the pipeline so it is re-fitted on the training part of every fold – no information leaks from the validation fold.
- **Scoring:** accuracy.

In [25]:
```
pipe = Pipeline([("scaler", StandardScaler()), ("knn", KNeighborsClassifier())])

param_grid = {
    "knn__n_neighbors": list(range(1, 26)),
    "knn__weights": ["uniform", "distance"],
    "knn__metric": ["euclidean", "manhattan"],
}

cv_iris = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
grid = GridSearchCV(pipe, param_grid, cv=cv_iris, scoring="accuracy", n_jobs=1)
grid.fit(X_train, y_train)                              # raw (unscaled) training data - the pipeline scales it
```

```
print("Best parameters (best_params_):", grid.best_params_)
print(f"Best cross-validated accuracy (best_score_): {grid.best_score_:.4f}")
```

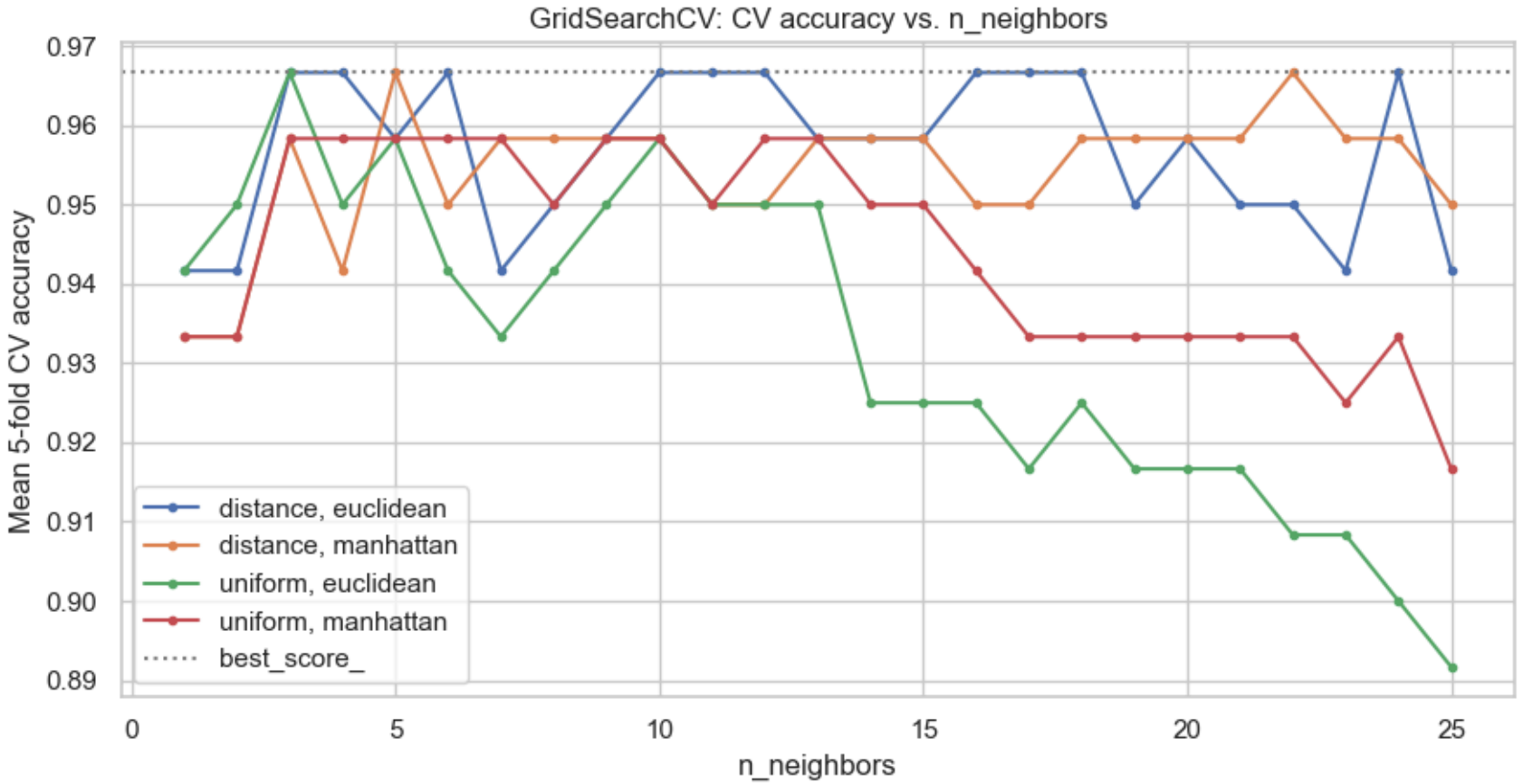
```
Best parameters (best_params_): {'knn__metric': 'euclidean', 'knn__n_neighbors': 3, 'knn__weights': 'uniform'}
Best cross-validated accuracy (best_score_): 0.9667
```

In [26]:
```
# Top combinations - note how many are tied
results_df = pd.DataFrame(grid.cv_results_)
n_tied = int((results_df["rank_test_score"] == 1).sum())
print(f"{n_tied} of {len(results_df)} combinations tie for rank 1 (GridSearchCV returns the first of them).\n")
display(results_df.sort_values(["rank_test_score", "param_knn__n_neighbors"])
        [["param_knn__n_neighbors", "param_knn__weights", "param_knn__metric", "mean_test_score", "std_test_score", "
        .head(8).round(4).reset_index(drop=True))
```

```
13 of 100 combinations tie for rank 1 (GridSearchCV returns the first of them).
```

	param_knn__n_neighbors	param_knn__weights	param_knn__metric	mean_test_score	std_test_score	rank_test_score
0	3	uniform	euclidean	0.9667	0.0167	1
1	3	distance	euclidean	0.9667	0.0167	1
2	4	distance	euclidean	0.9667	0.0167	1
3	5	distance	manhattan	0.9667	0.0167	1
4	6	distance	euclidean	0.9667	0.0167	1
5	10	distance	euclidean	0.9667	0.0167	1
6	11	distance	euclidean	0.9667	0.0167	1
7	12	distance	euclidean	0.9667	0.0167	1

In [27]:
```
# Cross-validated accuracy as a function of k for every (weights, metric) combination
fig, ax = plt.subplots(figsize=(9, 4.8))
for (w, m), part in results_df.groupby(["param_knn__weights", "param_knn__metric"]):
    part = part.sort_values("param_knn__n_neighbors")
    ax.plot(part["param_knn__n_neighbors"].astype(int), part["mean_test_score"], marker="o", ms=3, label=f"{w}, {m}"
ax.axhline(grid.best_score_, color="gray", ls=":", label="best_score_")
ax.set(title="GridSearchCV: CV accuracy vs. n_neighbors", xlabel="n_neighbors", ylabel="Mean 5-fold CV accuracy")
ax.legend(); plt.tight_layout(); plt.show()
```



2. Compare the Tuned Model with the Default KNN of Task 1(b)

In [28]:
```
# Default model: same pipeline with default hyper-parameters, scored with the same 5 folds for a fair comparison
default_cv = cross_val_score(pipe, X_train, y_train, cv=cv_iris, scoring="accuracy").mean()

best_knn = grid.best_estimator_                          # already re-fitted on the whole training set
y_pred_tuned = best_knn.predict(X_test)

comparison = pd.DataFrame({
    "Default KNN (k=5, uniform, euclidean)": {
        "5-fold CV accuracy (train)": default_cv,
        "Test accuracy": default_metrics["Accuracy"],
        "Test precision (macro)": default_metrics["Precision"],
```

```
        "Test recall (macro)": default_metrics["Recall"],
        "Test F1 (macro)": default_metrics["F1-score"]},
    "Tuned KNN (GridSearchCV)": {
        "5-fold CV accuracy (train)": grid.best_score_,
        "Test accuracy": accuracy_score(y_test, y_pred_tuned),
        "Test precision (macro)": precision_score(y_test, y_pred_tuned, average="macro"),
        "Test recall (macro)": recall_score(y_test, y_pred_tuned, average="macro"),
        "Test F1 (macro)": f1_score(y_test, y_pred_tuned, average="macro")},
}).round(4)
display(comparison)

print("Classification report – tuned KNN:\n")
print(classification_report(y_test, y_pred_tuned))
print("Confusion matrix (rows = actual, columns = predicted):")
print(confusion_matrix(y_test, y_pred_tuned))
```

	Default KNN (k=5, uniform, euclidean)	Tuned KNN (GridSearchCV)
5-fold CV accuracy (train)	0.9583	0.9667
Test accuracy	0.9333	0.9333
Test precision (macro)	0.9444	0.9444
Test recall (macro)	0.9333	0.9333
Test F1 (macro)	0.9327	0.9327

```
Classification report - tuned KNN:

                precision    recall  f1-score   support

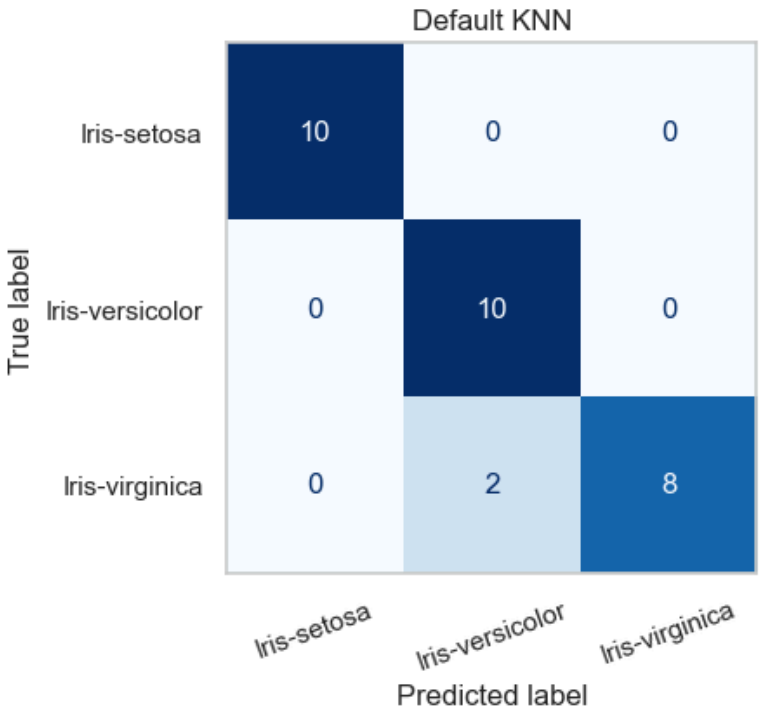
    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       0.83      1.00      0.91        10
 Iris-virginica       1.00      0.80      0.89        10

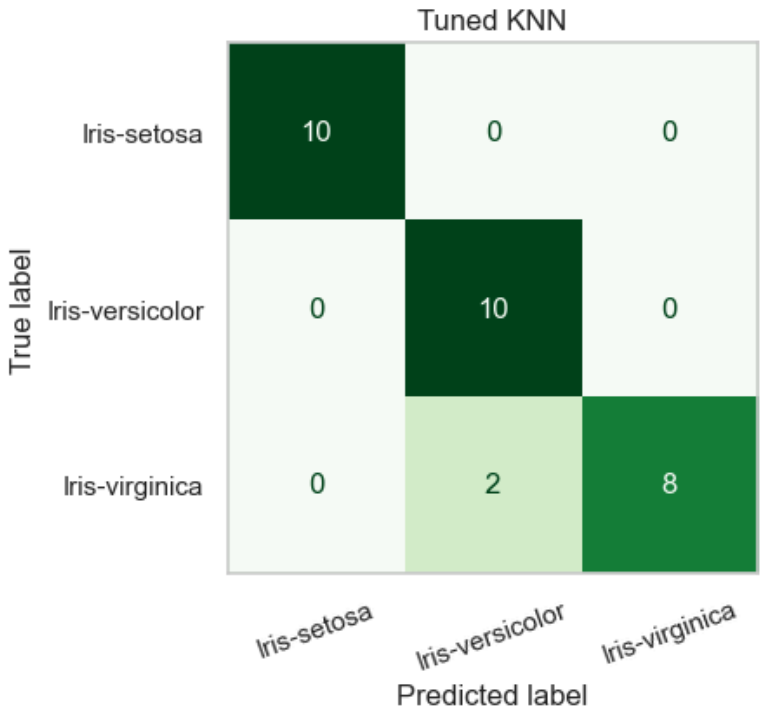
       accuracy                           0.93        30
      macro avg       0.94      0.93      0.93        30
   weighted avg       0.94      0.93      0.93        30

Confusion matrix (rows = actual, columns = predicted):
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
```

In [29]:

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap="Blues", ax=axes[0], xticks_rotation=20, colorbar=False)
axes[0].set_title("Default KNN"); axes[0].grid(False)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_tuned, cmap="Greens", ax=axes[1], xticks_rotation=20, colorbar
axes[1].set_title("Tuned KNN"); axes[1].grid(False)
plt.tight_layout(); plt.show()
```





3. Findings and Insights

- **Best parameters:** `n_neighbors = 3`, `weights = 'uniform'`, `metric = 'euclidean'`, with a **best cross-validated accuracy of 0.9667**.
- **Tuning improved the cross-validated accuracy** from **0.9583** (default k = 5) to **0.9667** – one extra correctly classified flower per 120.
- **Test accuracy did not change: 0.9333 for both models.** The tuned model makes exactly the same two errors (virginica predicted as versicolor). The test set has only 30 flowers, so a gain of 0.8 percentage points in cross-validation is far too small to show up there.
- **Many settings are equally good:** 13 of the 100 combinations tie for the top CV accuracy, and 84 % of the settings with k between 3 and 16 reach at least 0.95 (range 0.925–0.967). Only very large k hurts – the weakest setting drops to 0.892 at k = 25 – while the whole grid spans just 0.892–0.967. On a clean, well-separated dataset like Iris, KNN is therefore **fairly insensitive to hyper-parameters**; tuning mainly confirms that the default was already sensible.

- **Where tuning matters more:** on noisier, higher-dimensional data (such as the SMS task above) hyper-parameters change the result a lot – there GridSearchCV (and the decision threshold) gave a clear improvement.
- **Good practice followed:** tuning used only the training data with 5-fold CV, the scaler sits inside the pipeline (no leakage), and the test set was used once for the final comparison.

Overall Summary

Task	Method	Key result
1(a) K-Means (Mall customers)	Elbow method + silhouette → **k = 5**, standardised income & spending	**Inertia = 65.57** (scaled), silhouette 0.555; 5 clear customer segments, highest-value group = high income + high spending (39 customers)
1(b) KNN (Iris)	k = 5, standardised features, 80/20 stratified split	**Accuracy 93.3 %**, macro F1 0.933; only virginica ↔ versicolor confusion (2 errors)
1(c) Naive Bayes (SMS spam)	CountVectorizer + MultinomialNB; imbalance techniques compared	Baseline spam precision 0.959 / recall 0.901 → tuned model with threshold: **precision 0.983, recall 0.901, F1 0.940** (ham blocked 5 → 2)
2(a) GridSearchCV (KNN, Iris)	100 combinations, stratified 5-fold CV	Best: k = 3, uniform, euclidean; **CV accuracy 0.9583 → 0.9667**; test accuracy unchanged at 0.9333

Key lessons: (1) always scale features for distance-based algorithms (K-Means, KNN); (2) on imbalanced data judge models by the precision and recall of the minority class and choose the trade-off from the business cost of each error; (3) tune hyper-parameters with cross-validation on training data only, and be careful interpreting small differences on small test sets.